Simcenter Testlab Introduction to Automation for Beginning Programmers

Linda Lee (linda.lee@siemens.com)

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

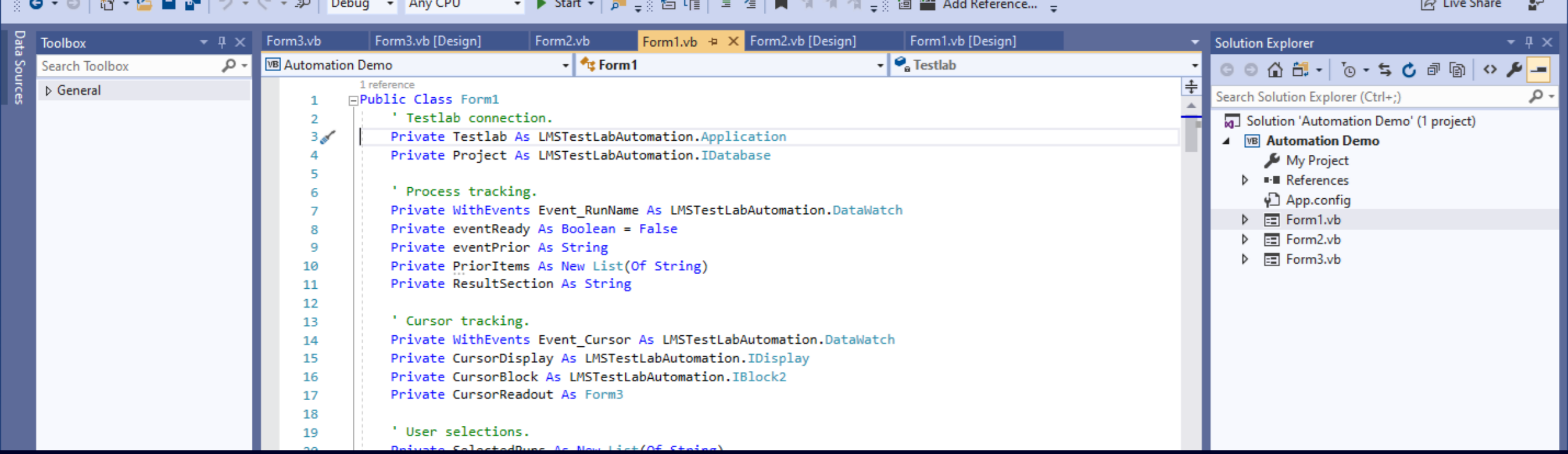
SIEMENS

Agenda

- Automation Overview
  - Subroutines, Functions, Properties, and Methods
  - Types, Objects, Instances, and Variables
  - Conditions and Iterations

- Walkthrough
  - Connecting to Simcenter Testlab
  - Reading the project contents
  - Creating new data

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software



Automation Overview

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

What Is Windows Automation?

- A Microsoft technology, not exclusive to Simcenter Testlab
  - Used by many different Windows Programs, including Microsoft Office

- How does it work?
  - The software package exposes features to be accessible by external programs
  - The application acts as a “server,” exposing the features to a “client” program

- The automation capability is *not* a script running inside Simcenter Testlab.
  - It is an external program running alongside Testlab and interacting with it.
  - The actual code can be in any programming language that is capable of using type libraries and implementing the COM object model.

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

Subroutines and Functions

It is possible to define a task using a self-contained region of code, which can be ‘called’ to have its code run before the rest of the program resumes.

- If this code creates and returns a value, it is known as a “function.”
- Otherwise, it is known as a “subroutine” or “sub.”

Subroutines and functions can take “arguments,” which are a set of inputs to convey information. Depending on the programming language or keyword options, these can be *by value* to create a temporary variable with the same information, or *by reference* to allow the input argument’s underlying value to be changed.

$$f(x, y) \coloneqq x^2 + y^2$$
$$a = 3,\ b = 4:$$
$$f(a, b) = 25$$

```
Function SumOfSquares(x, y)
  Dim sum As Integer
  sum = x^2 + y^2
  Return sum
End Function
```

```
Dim a, b, sumSq As Integer
a = 3
b = 4
sumSq = SumOfSquares(a, b)
```


Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

Subroutines and Functions

```
Sub Main()
  Dim input, output As Integer
  input = 3
  output = AddTwo(input)
  input = output
  output = AddTwo(input)
End Sub

Function AddTwo(number) As Integer
  number = number + 1
  AddTwo = number + 1
End Function
```

If the argument to AddTwo was…					
ByVal (by value)			ByRef (by reference)		
input	number	output	input	number	output
3	3 -> 4	5	3 -> 4	3 -> 4	5
5	5 -> 6	7	5 -> 6	5 -> 6	7

By Value, this only changes the local variable “number.”

By Reference, the variable “number” is a direct connection to the source data, which would have its value increased by 1.

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

Basic Data Types

How can you represent a number-type value?

- **Integer:** like a mathematical integer, this represents a signed (±) whole number.
- **Double:** meaning “double precision,” this works like scientific notation, storing a number’s significant digits, sign, and the power of 10 that determines the location of its decimal point.
- **Float / Single:** meaning “floating decimal point,” this functions the same way as a double, but holds half as many significant digits. This type is not used by Testlab.

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

Basic Data Types

How can you represent a text-type value?

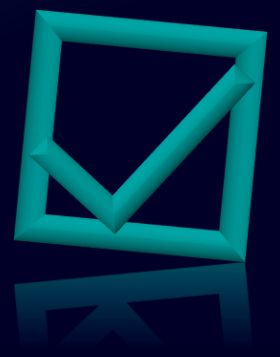
- **Char:** meaning “character,” this is a single symbol, such as a letter, numeral, punctuation mark, or whitespace.
- **String:** this is an array of characters ‘strung together’ to form words, sentences, paragraphs, and so on. This datatype is how text is typically represented.
  - Some programming languages treat this like a primitive (basic) datatype with a default value of the empty string “”, while other languages treat it as an object type that is null-valued until instantiated, or as an array of characters.
  - Different programming languages have different ways of representing strings, such as between single or double quotes, and of combining strings, such as with the & or + operators, or dedicated functions such as Matlab’s strcat().

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

Basic Data Types

How can you represent a logical value?

- **Boolean:** this type can have two possible values: “true” or “false.”



How can you represent multiple values of the same type?

- **Array:** this forms a matrix of the specified datatype, with values that can be accessed using indices, using base zero or base one depending on the language. Many languages require the matrix to be rectangular.

```
Dim myIntegers(2, 1) As Integer
myIntegers(0, 0) = 123        myIntegers(0, 1) = 321
myIntegers(1, 0) = 456        myIntegers(1, 1) = 654
myIntegers(2, 0) = 789        myIntegers(2, 1) = 987
```

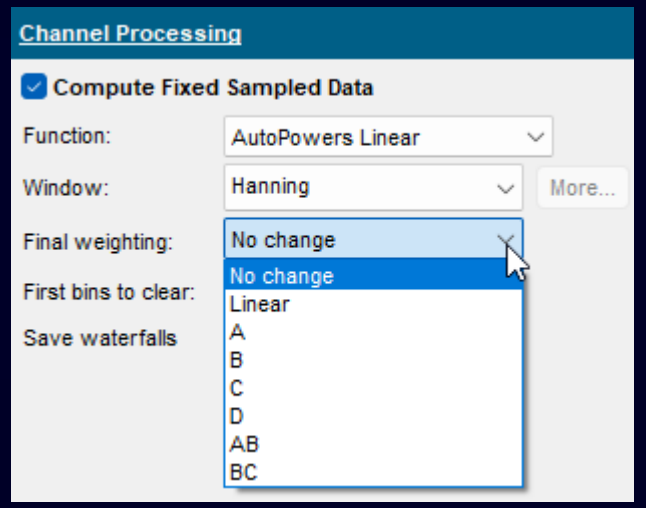
$$\begin{array}{c|cc} & 0 & 1 \\ \hline 0 & 123 & 321 \\ 1 & 456 & 654 \\ 2 & 789 & 987 \end{array}$$

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

How does this apply to Testlab Automation?

These primitive types are the ‘building blocks’ behind more the complex objects in the type library, and can often be retrieved and used from the object properties.

```
Dim weighting As LMSTestLabAutomation.Enumerate
Set weighting = Testlab.ActiveBook.FindDataWatch( _
                    “TMon\/Signature\/Acoustic\/WeightingType”).Value
Dim weightingName As String
weightingName = weighting.LocalValue
Dim weightingIndex As Integer
weightingIndex = weighting.IntValue
```



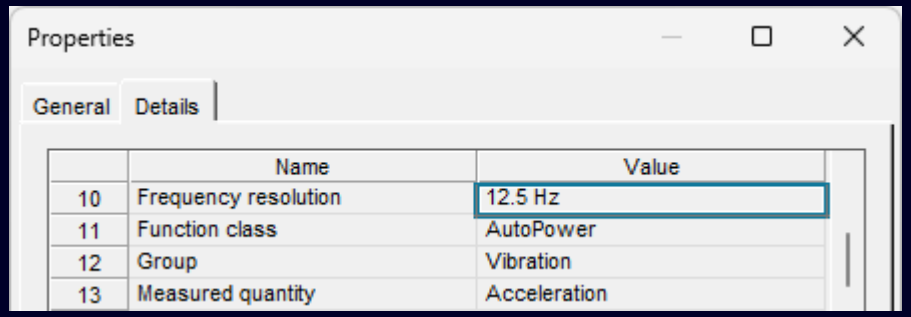
 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

How does this apply to Testlab Automation?

Even when some object types don’t have direct properties for access to the values, the automation library still provides other ways to retrieve the information.

```
Dim resolution As LMSTestLabAutomation.IScalarVariable
Set resolution = myBlock.Properties( “Frequency resolution” )
Dim real, imag As Double
resolution.GetValue( real, imag )
Dim unit As String
unit = Testlab.UnitSystem.Label( resolution.Quantity )
real = Testlab.UnitSystem.MKSToUserValue( resolution.Quantity, real )
MsgBox( real & “ ” & unit )
```



Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS

Objects and Instances

An “object type” is the scaffold for a structure that can hold data, as well as to define related Properties and Methods, which are a set of subroutines and functions for managing that data.

The main difference between Properties and Methods is a matter of convention:

- A Property is typically used to get or set a defined aspect of the data, and reading its value should not enact a change in the underlying data.
- A Method is typically used to perform an ‘action,’ or to retrieve data that might require dynamic computation rather than simply reading a value.

Some programming languages may have syntax differences between the two.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Objects and Instances

An object can be considered as a way to represent a concept as a defined structure.

- Example: a “Basketball” object type could be defined as:
  - Property: Radius [number]
  - Property: Color [text string]
  - Property: Position [X, Y, Z number triplet]
  - Method: Bounce [action to alter the Position property]
  - Method: Inflate [action to alter the Radius property]

This object definition is a scaffold of how data is structured: it is possible to define relations between calculated properties (e.g. a Property for Diameter, which returns Radius * 2), but this definition itself does not yet correspond to a specific item.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Objects and Instances

An “instance” is a value that follows an object type’s structure.

- Example:
  - A Basketball-type variable named “favorite” containing the following instance:
    - Properties: Radius = 1 foot, Color = “bright orange”, Position = 0, 0, 2
    - Calling the Bounce method on this object could change its Z coordinate.
  - A Basketball-type variable named “spare” containing the following instance:
    - Properties: Radius = 0.5 foot, Color = “faded yellow”, Position = 100, 50, 0
    - Calling the Inflate method on this object could increase its Radius.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Objects and Instances

A “variable” is a user-defined place to hold a value.

- Declaring a variable gives it both a name and a type.
  - Effectively, this creates a memory slot of the right shape to hold that type.
  - For objects, the value is ‘Nothing’ (null) until it is assigned or constructed.
  - For base types, a default value is automatically assigned.
- Constructing or assigning an object variable gives it an instance as its value.
  - This places an instance into the memory slot; the properties and methods of the instance can then be used from the variable.
    - Assigning: you can store a result created by another function or property.
    - Constructing: some, but not all, object types allow a new instance to be created; many programming languages use the keyword “New” for this.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

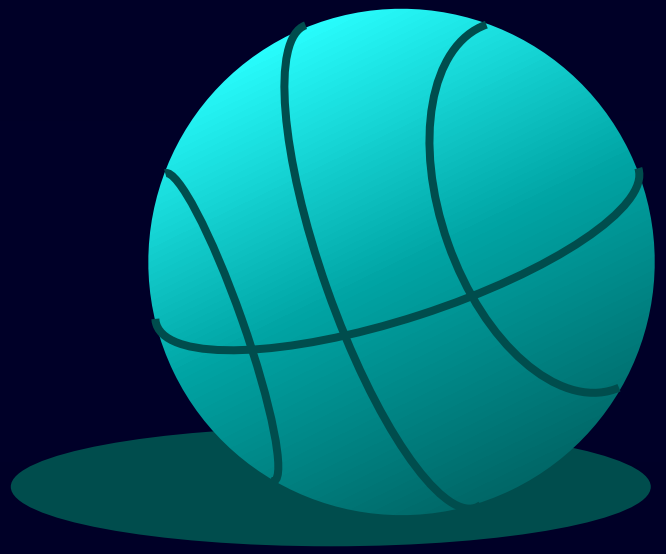
Objects and Instances

Object Definition

- A variable's Class or Type
- Defines what properties are available
- Defines which methods are available and what they do
- 'Blueprint'

Instance of an Object

- Can be assigned as a variable's value
- Property values can be accessed
- Methods can be called or invoked to perform actions
- 'Constructed'



Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

How does this apply to Testlab Automation?

The main connection to Simcenter Testlab requires a variable of the object type LMSTestLabAutomation.Application, which is then ‘constructed’ to create an instance. The link to Testlab isn’t created until the “New” constructor is used.

In VBA and Visual Basic, the “Dim” keyword creates a new variable.

Dim Testlab As LMSTestLabAutomation.Application Declares the name & type.

Set Testlab = New LMSTestLabAutomation.Application Constructs a new instance.

Dim Project As LMSTestLabAutomation.IDatabase Declares the name & type.

Set Project = Testlab.ActiveBook.Database Assigns the retrieved instance.

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

How does this apply to Testlab Automation?

When retrieving values from properties or ports, there might not always be an ‘instance’ present – for example, some worksheets don’t load their setting values until they’ve been accessed – so it is important to check for null values.

```
Dim channelList As LMSTestLabAutomation.IData
Set channelList = Testlab.ActiveBook.FindDataWatch( _
                                    “TMon\/Signature\/AcousticChannelIds”).Value
If channelList Is Nothing Then
  MsgBox( “No channels found - please load the data before running this program!” )
Else
  MsgBox( channelList.AttributeMap.Count )   If null, this would have thrown an error.
End If
```

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

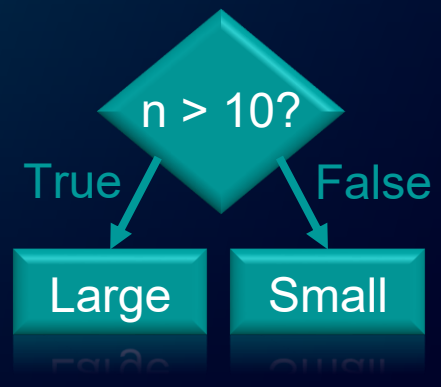
SIEMENS

Conditional and Iteration Statements

It is useful for programs to be able to take different paths or repeat a number of times depending on what sort of data is encountered along the way. While each programming language may have a different syntax, the general idea of conditional statements and iteration loops are present across most languages.

Conditions: evaluate a true/false or a match to find which code to execute.

```
If (n > 10) Then
  MsgBox( “n is large”)
Else
  MsgBox( “n is small”)
End If
```




```
Select Case n
  Case 1: MsgBox( “one” )
  Case 2: MsgBox( “two” )
  Case Else: MsgBox ( “n is ” & n )
End Select
```


Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS

Conditional and Iteration Statements

Iteration: repeat the nested code a fixed number of times, for each element of a collection, or until a condition is true.

Start End Increment

```
For n = 1 To 9 Step 2
  MsgBox( “Odd number: ” & n )
Next n
```

Collection

```
For Each n In {2, 4, 6, 8, 10}
  MsgBox( “Even number: ” & n )
Next n
```

```
Dim n As Integer
n = 3
```

Condition

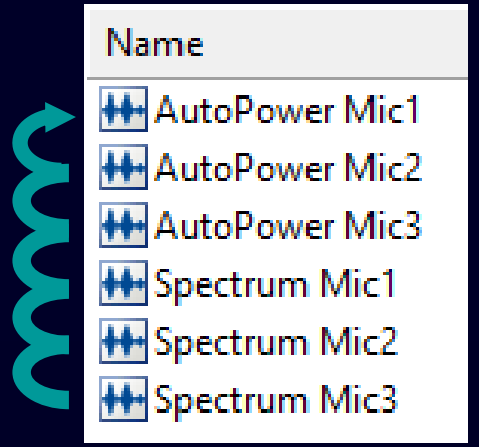
```
Do Until n > 10
  MsgBox( “Multiple of three: ” & n )
  n = n + 3
Loop
```


Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS

How does this apply to Testlab Automation?

When dealing with varying project setups and datasets, conditions allow checking for validity or handling data in different ways, while iteration allows multiple channels or blocks to be processed without needing to know in advance how many there are.

```
Dim block As LMSTestLabAutomation.IBlock2
For Each blockPath In Project.ElementNames( “Section 1/Folder of Blocks”, 1 )
  Set block = Project.GetItem( blockPath )
  If block.Properties( “Function class” ) = “AutoPower” Then
    MsgBox( “Found an AutoPower to process: ” & block.Label )
  End If
Next
```



 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

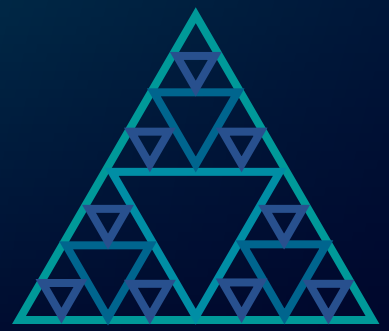
SIEMENS

Recursion

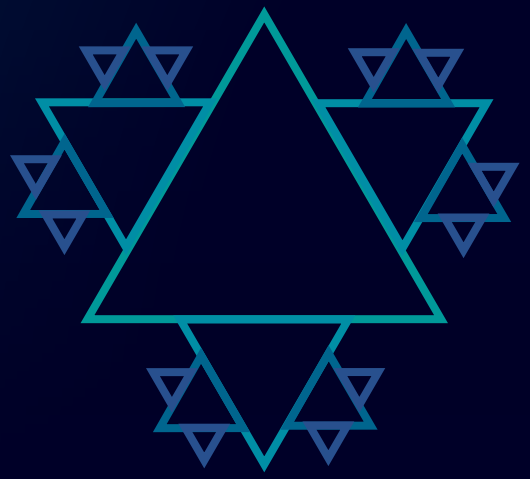
Recursion is a way of repeatedly applying code by having a subroutine call itself. Unlike iteration, which performs one repetition after another like a linear ‘chain’, a recursion can be nested inside of itself like a ‘fractal.’

Iteration:

Recursion:



or



Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

How does this apply to Testlab Automation?

Recursion is useful for browsing properties in properties, and folders in folders.

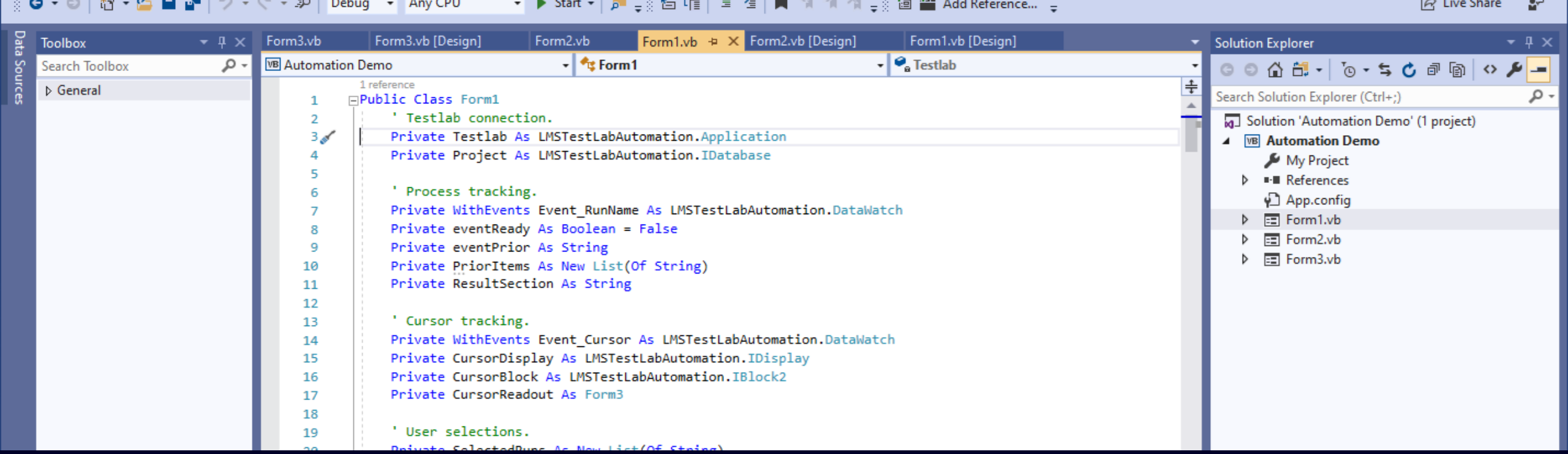
```
Function FirstAvailableBlock(ByVal folderPath As String)
  Dim block As LMSTestLabAutomation.IBlock2
  For Each elementPath In Project.ElementNames( folderPath, 1 )
    Select Case Project.ElementType( elementPath )
      Case “Section”, “Run”, “Folder”
        Set block = FirstAvailableBlock( elementPath )
        If block IsNot Nothing Then Return block
      Case “Block” : Return Project.GetItem( elementPath )
    End Select
  Next
End Function
```

Iteration:
- Element 1
- Element 2
- …

Recursion:
- Section
  - Run
    - Folder
      - Folder
    - Folder
      - Folder
  - Run
  - …

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

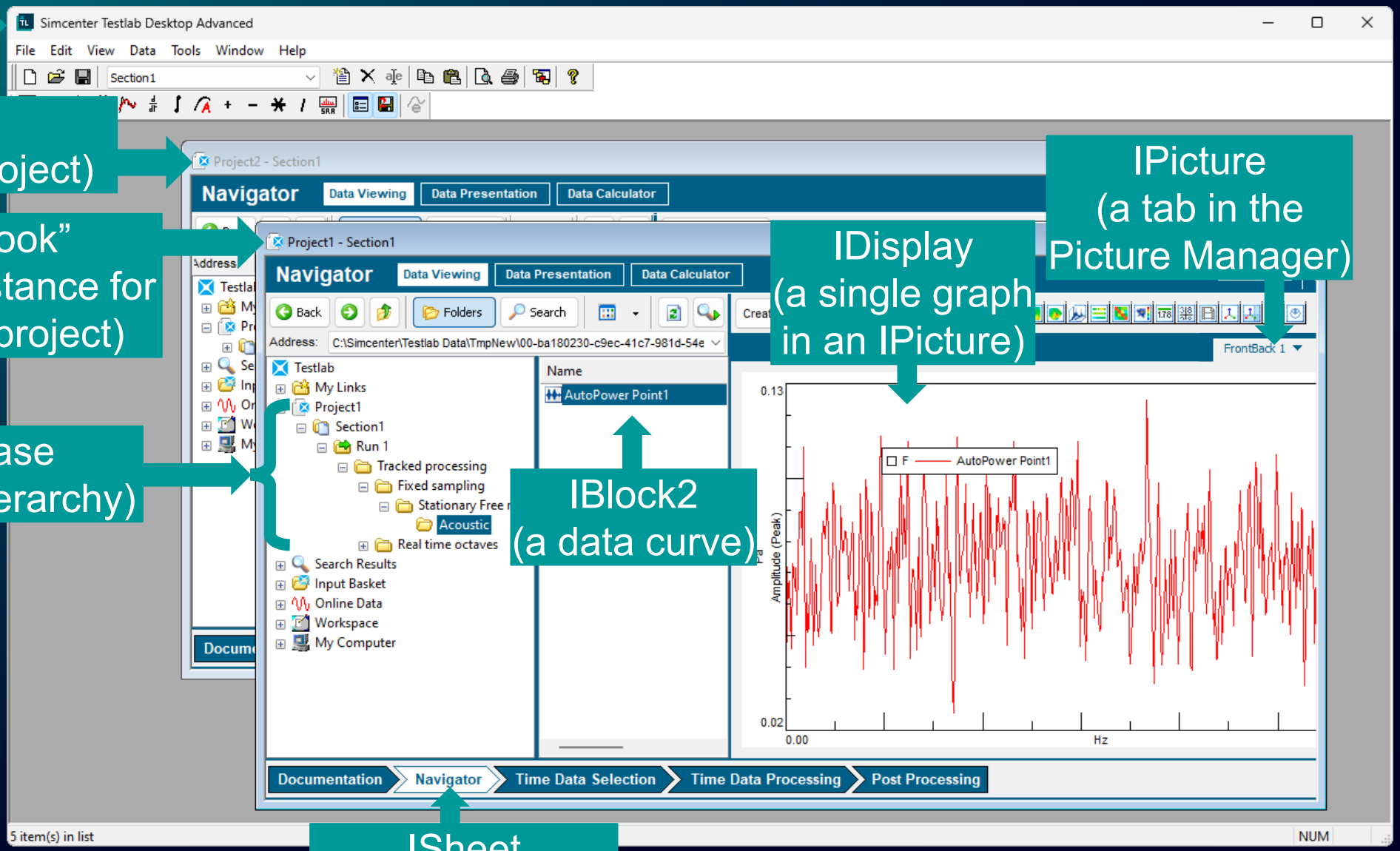


Common Datatypes of the Simcenter Testlab Object Model

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

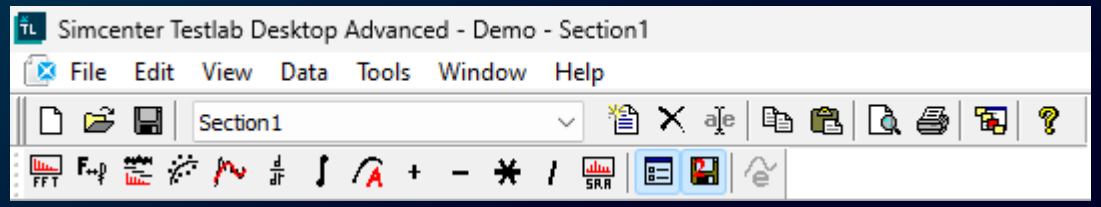
Common Simcenter Testlab Object Types



Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.Application



Properties:

- Name: a String representing the current workbook type.
- ActiveBook: the IBook that is ‘on top’ in the application.
- Database(*filename*): the IDatabase of a project that isn’t open.
- UnitSystem: the IUnitSystem defined in the Configuration & Unit System tool.

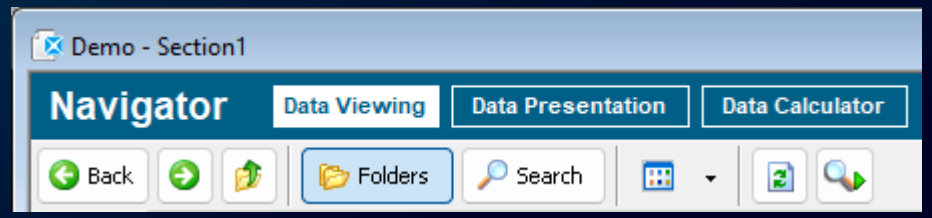
Methods:

- NewProject(*[template]*): creates a new project in Simcenter Testlab; a template and/or workbook can be specified.
- OpenProject(*filename*): opens the specified project in Simcenter Testlab.
- Quit: closes Simcenter Testlab; any currently open unsaved projects will show a dialog message confirming whether to close them or not.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.IBook



Properties:

- ActiveProjectWatch: a Watch linked to the current project name.
- ActiveSheetWatch: a Watch linked to the current sheet name.
- SheetOnTop: the String name of the sheet to switch to (write-only).

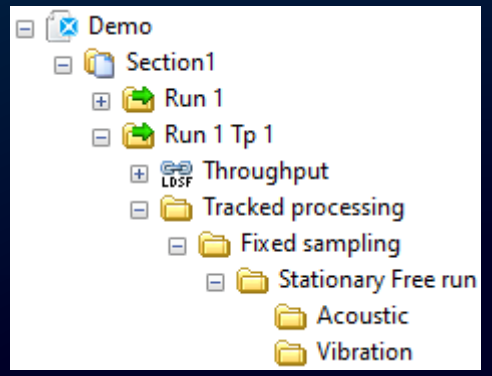
Methods:

- Database: returns the IDatabase of the project.
- FindDataWatch(path): returns a DataWatch linked to the specified port.
- Save: saves the project.
- Close: closes the project; if unsaved, the user will receive a confirmation dialog.
- CloseWithoutSave: closes the project without a confirmation dialog.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.IDatabase



Properties:

- ElementNames(*path, [returnAsPathToTop]*): a map of the element names located in the specified path; if the second argument is 1, the full paths are returned.
- ElementType(*path*): a String of the datatype of the item at the specified path.
- SectionNames: a map of the project's section names.

Methods:

- AddItem(*path, name, item, [properties], [overwrite]*): writes the provided item.
- AddFolder, AddSection, & AddRun(*path, name*): creates a folder, section, or run.
- GetItem(*path*): retrieves the data item at the specified path.
- GetProperties(*path*): retrieves the property map of the item at the specified path.
- Save: saves the project's database.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.AttributeMap

	Name	Value
1	Average type	Energy average
2	Bandwidth	6400 Hz
3	Channel group	Vibration
4	Channel id	1
5	Channel name	C1
6	Created	2024 05 20 Mon 09:16:18
7	Creation time stamp	2024 05 23 Thu 13:08:37
8	DOF ID	Point1:+X

This object is a collection of keys and corresponding values, functioning as a list when using integer indices, or as a map / dictionary when using String indices.

Properties:

- Count: an integer for the number of items in the map.
- KeyNames: an AttributeMap where the indices are integers and the values are the String indices of the current map.
- Item(*index*): the item associated with the index.

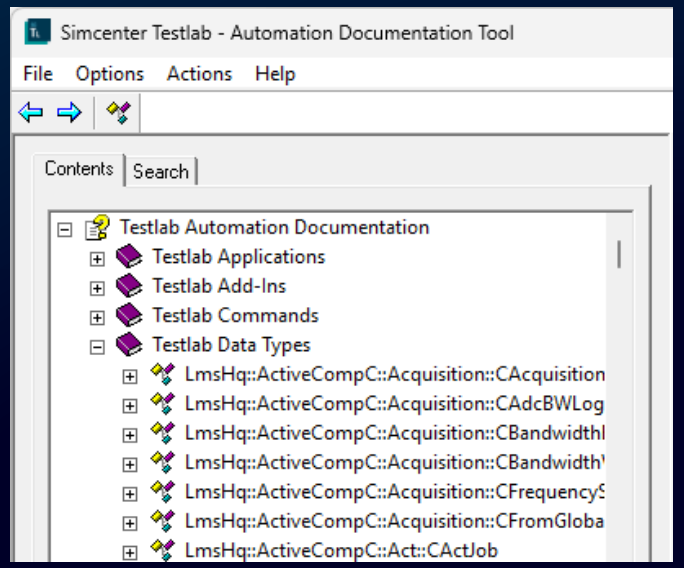
Methods:

- Add(*index, value*): adds an index and assigns it the value.
- Replace(*index, value*): changes the value for an existing index to the new value.
- Remove(*index*): removes the index and its associated value.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.IData



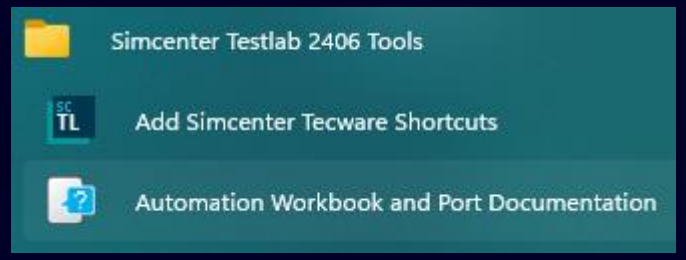
This is the base type of many LMSTestLabAutomation objects.

Properties:

- AttributeMap: a map of the data's values.
- Type: a String describing the IData's datatype, which can be looked up in the Automation Workbook and Port Documentation tool.

Methods:

- Edit(*path, value*): returns a modified copy for which the path key is assigned the new value. Not all IData are editable; uneditable ones will raise an error.
- Item(*path*): returns the item in its map associated with the path index.



 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

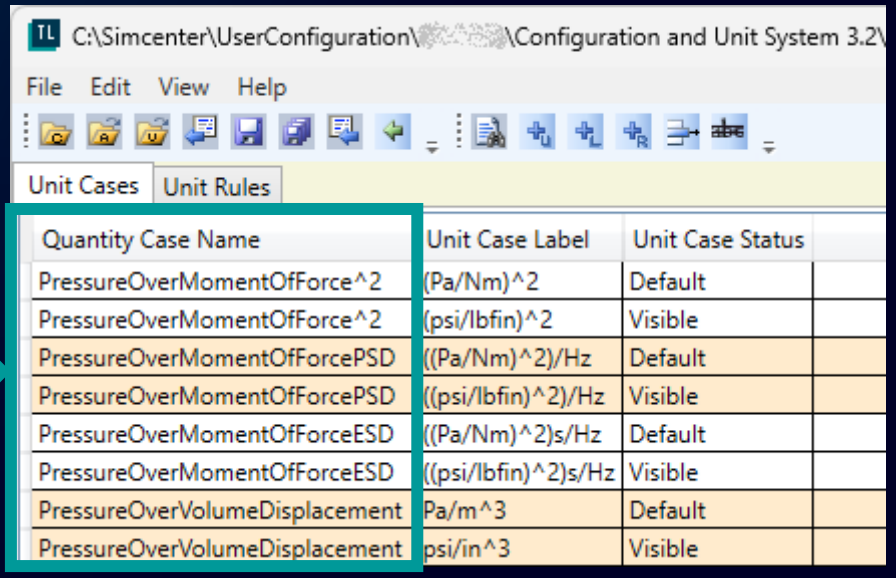
SIEMENS

LMSTestLabAutomation.IQuantity

This represents a quantity of the unit system.

Properties:

- QuantityType: the String quantity type, such as Time or Pressure.



Quantity Case Name	Unit Case Label	Unit Case Status
PressureOverMomentOfForce^2	(Pa/Nm)^2	Default
PressureOverMomentOfForce^2	(psi/lbfin)^2	Visible
PressureOverMomentOfForcePSD	((Pa/Nm)^2)/Hz	Default
PressureOverMomentOfForcePSD	((psi/lbfin)^2)/Hz	Visible
PressureOverMomentOfForceESD	((Pa/Nm)^2)s/Hz	Default
PressureOverMomentOfForceESD	((psi/lbfin)^2)s/Hz	Visible
PressureOverVolumeDisplacement	Pa/m^3	Default
PressureOverVolumeDisplacement	psi/in^3	Visible

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS

LMSTestLabAutomation.IUnitSystem

This class provides ways to use IQuantities for unit conversions.

Properties:

- QuantityByLabel(*label*): the IQuantity associated with the unit label.
- QuantityTime, QuantityPressure, …: the IQuantity for the known quantity.

Methods:

- Label(*quantity*): retrieves the String unit label, such as “s” or “Pa”.
- MKSToUserValue, UserToMKSValue(*quantity, number*): convert units.
- MKSToUserValues, UserToMKSValues(*quantity, array*): convert units.
- ValueTodBValue, dBValueToValue(*quantity, number*): convert to/from decibel.
- ValuesTodBValues, dBValuesToValues(*quantity, array*): convert to/from decibel.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.IScalar & IScalarVariable

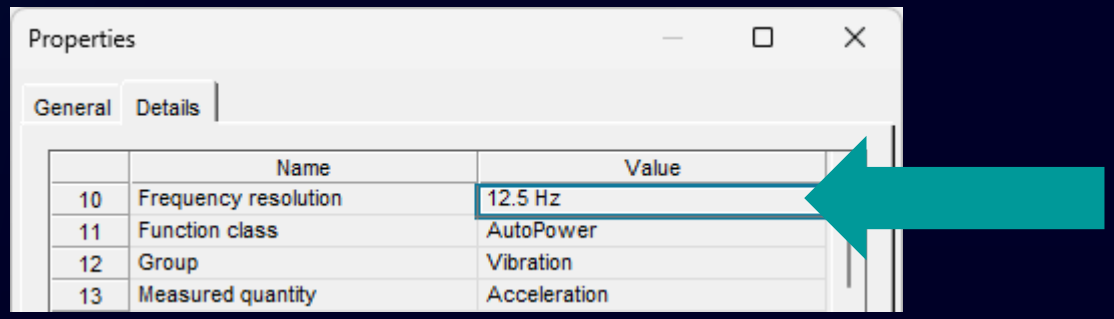
This represents a complex value paired with a unit.

Properties:

- Quantity: the IQuantity of the unit.

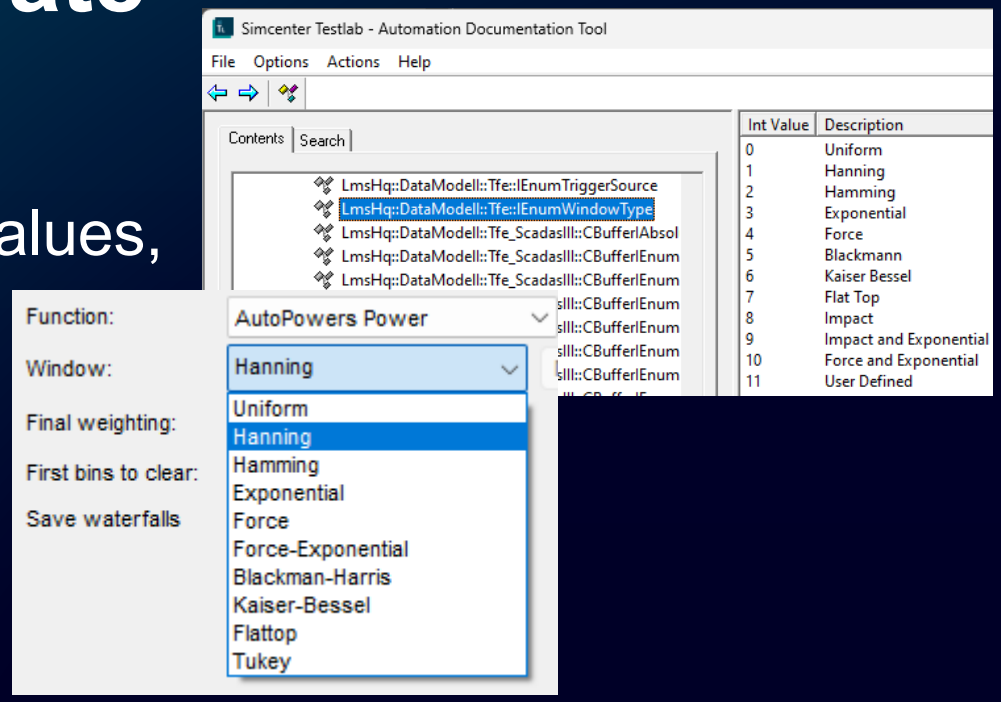
Methods:

- GetValue(*ByRef real, ByRef imag*): alters the input arguments to assign the real and imaginary components of its value to them, in MKS units.



Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS

LMSTestLabAutomation.Enumerate



This represents an option from a predefined set of values, such as the possible window types.

Properties:

- LocalValue: the String name of the option.
- IntValue: the Integer index of the option.
- StringIdValue: the String name of the option, with a simplified format.
- LocalRange, Range, StringIdRange: a map of the available options.

Methods:

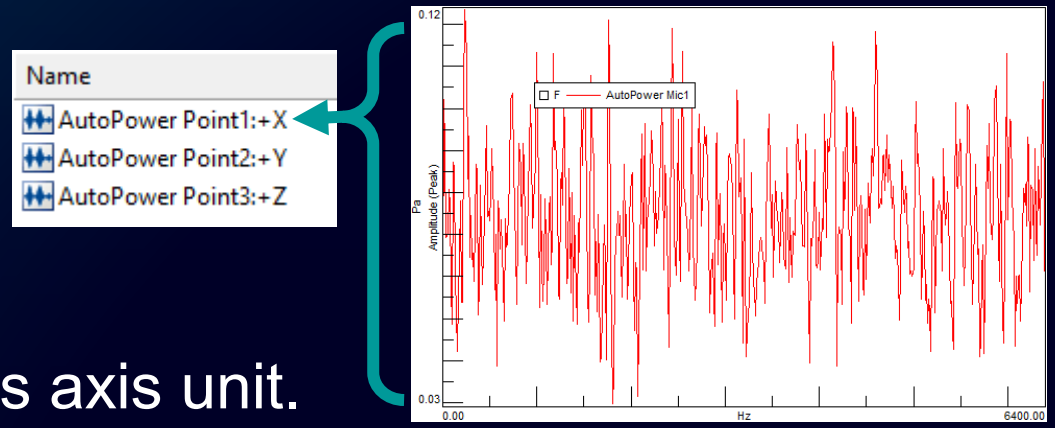
- ReplaceLocalValue, ReplaceIntValue, ReplaceStringIdValue(*value*): returns another Enumerate of the same type, but with the specified different option.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.IBlock2

This represents a data curve of y-values against one or more x axes.



Properties:

- Label: the block name.
- Properties: the block's property map.
- XQuantity(*[index]*), YQuantity: the IQuantity of its axis unit.
- XValues(*[index]*), YValues, RealYValues: arrays of the data values.

Methods:

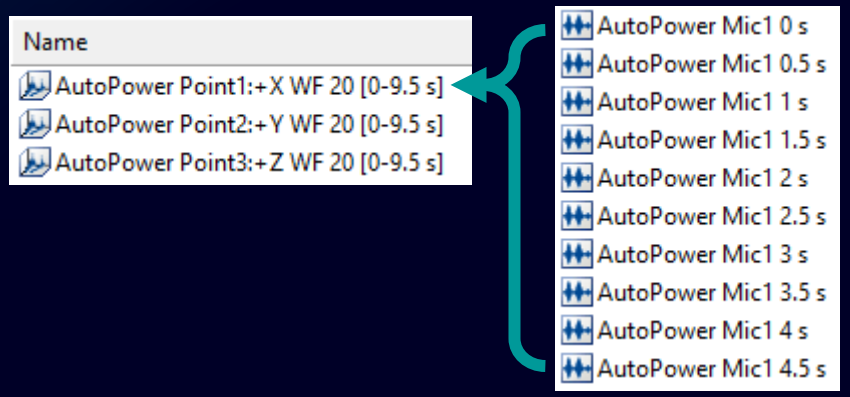
- ReplaceXQuantity(*quantity, [index]*), ReplaceYQuantity(*quantity*)
- ReplaceXDoubleValues, ReplaceXIntValues, ReplaceYDoubleValues, ReplaceYComplexValues(*values*), …: returns a block with edited values.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.IWaterfall

This represents a collection of blocks along a z axis in a waterfall / colormap.



Properties:

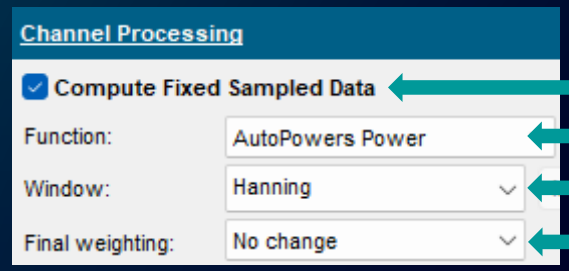
- Block(*[index]*): the specified block within the waterfall.
- BlockCount: the number of blocks in the waterfall.
- Properties: the property map of the waterfall.
- ZCount: the number of z axes.
- ZQuantity(*[index]*): the IQuantity of the specified z axis' unit.
- ZValues(*[index]*), RealZValues(*[index]*): the z value array for the specified axis.

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

LMSTestLabAutomation.Watch & DataWatch

These represent ports for settings and buttons in the software. DataWatch ports are typically retrieved from the ActiveBook's FindDataWatch method using path names that can be found in the Automation Workbook and Port Documentation tool.



TMon\Signature\Processing\Acoustic_On
TMon\Signature\Acoustic\FunctionType
TMon\Signature\Acoustic\WindowingType
TMon\Signature\Acoustic\WeightingType

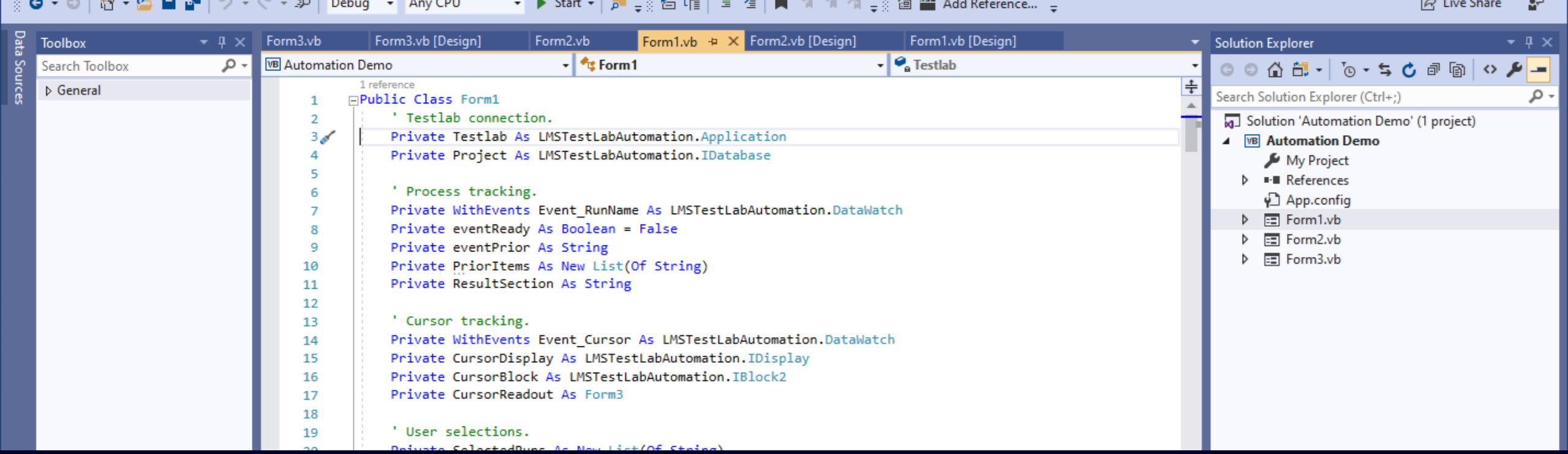
Properties:

- Data: the current value.

Events:

- ValueChanged(*newValue*): occurs when the value of the port changes. Programming languages that support COM object events can use this event handle to enact a subroutine when a value changes, such as for monitoring the port of the measurement status in order to perform a task after the new measurement is completed.

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS



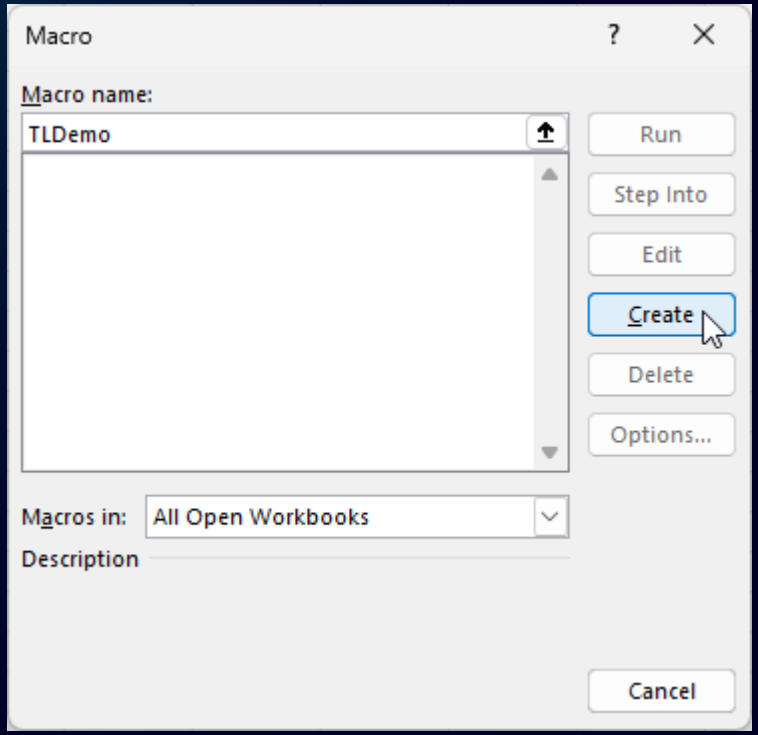
Walkthrough

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Creating a macro

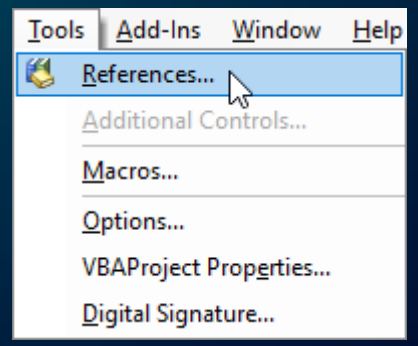
In the View ribbon of Microsoft Excel, select the Macros dropdown, and open the View Macros dialog. Then, enter a name for the new macro and select Create.

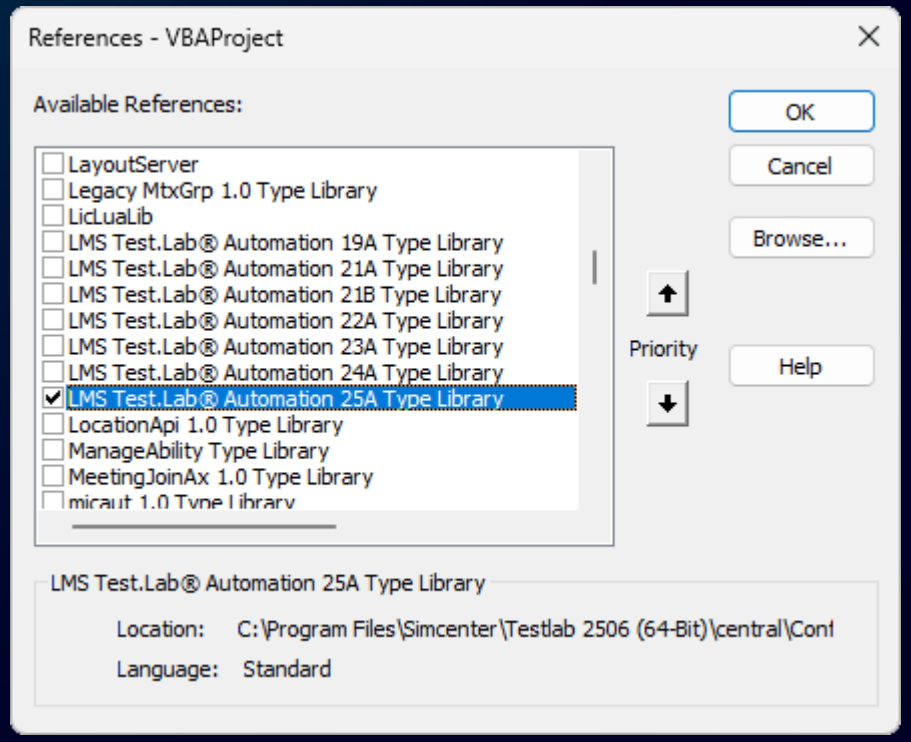


Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS

Connecting to Testlab

To load the Simcenter Testlab object model, select Tools > References from the toolbar. Check the box for the LMS Test.Lab Automation Type Library with a version corresponding to the revision of Testlab your automation is to be used with.





 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Connecting to Testlab

The LMSTestLabAutomation.Application object represents the Simcenter Testlab application. It has a constructor (allows the New keyword) which does the following:

- If Testlab Classic is currently open, it will connect to the existing application.
  - If multiple instances are open, it will only connect to one of them.
- If Testlab Classic is not open, it will load it as a background process with no current project. This can be moved to the foreground via the methods .OpenProject(*ProjectPathName*) and .NewProject(*[TemplatePathName]*).

```
Dim Testlab As LMSTestLabAutomation.Application

Sub TLDemo()
  Set Testlab = New LMSTestLabAutomation.Application
  If Testlab.Name = "" Or Testlab.Books.Count = 0 Then
    Call Testlab.OpenProject("C:\Simcenter\Testlab Data\Demo.lms")
  End If
End Sub
```

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Reading the project contents

The .ActiveBook property of the Application object returns an IBook instance that refers to the application's active workbook, which includes the project file data and the current settings in the application. The .Database property of the IBook further returns the IDatabase of the embedded project data, such as sections, runs, folders, blocks, and waterfalls, in a hierarchical format.

```
Dim Testlab As LMSTestLabAutomation.Application
Dim Project As LMSTestLabAutomation.IDatabase

Sub TLDemo()
  Set Testlab = New LMSTestLabAutomation.Application
  If Testlab.Name = "" Or Testlab.Books.Count = 0 Then
    Call Testlab.OpenProject("C:\Simcenter\Testlab Data\Demo.lms")
  End If

  MsgBox Testlab.Name & vbNewLine & Testlab.ActiveBook.ActiveProjectWatch.Data
  Set Project = Testlab.ActiveBook.Database
End Sub
```

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Reading the project contents

Items can be retrieved from the IDatabase using their full path through the hierarchy, similar to accessing a folder or file in a Windows File Explorer by using a filepath.

If a project's contents aren't known beforehand, it can be useful to write a loop or recursion that browses deeper into the contents based on item type.

```
Sub BrowseRecursively(ByVal path As String, ByRef results As String)
  For Each elempath In Project.ElementNames(path, 1)
    Select Case Project.ElementType(elempath)
      Case "Section", "Run", "Folder"
        Call BrowseRecursively(elempath, results)
      Case "Block"
        results = results & vbNewLine & elempath
    End Select
  Next
End Sub
```

```
Dim pathsToBlocks As String
pathsToBlocks = "List of block paths:"
For Each section In Project.SectionNames
  Call BrowseRecursively(section, pathsToBlocks)
Next
MsgBox pathsToBlocks
```


Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

Reading the project contents

Once the path of an endpoint datatype such as a Block or Waterfall is known, it can be retrieved using the .GetItem(*PathToTop*) method of the IDatabase.

From there, the properties of the specific interface, such as IBlock2 for blocks, allow retrieval of the data values and properties.

```
Dim blockPath As String
blockPath = Split(pathsToBlocks, vbNewLine)(1)

Dim block As LMSTestLabAutomation.IBlock2
Set block = Project.GetItem(blockPath)

Dim xs, ysReal, ysComplex As Variant
xs = block.XValues(0)        ' Double()  1D Array
ysReal = block.RealYValues ' Double()  1D Array
ysComplex = block.YValues  ' Double(,) 2D Array
```

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Values and units

Values are stored using the internal standardized MKS unit system. These can be converted to the user default units or decibel format using the .UnitSystem property of the Application object, which returns an IUnitSystem with conversion methods.

```
Dim x, yReal, yImag, yAmp, yPhase, yDB As Double
x = xs(0)
yReal = ysComplex(0, 0) ' or yReal = ysReal(0)
yImag = ysComplex(0, 1)
yAmp = (yReal ^ 2 + yImag ^ 2) ^ 0.5
If yReal = 0 Then yPhase = Pi / 2 Else yPhase = Math.Atn(yImag / yReal)

x = Testlab.UnitSystem.MKSToUserValue(block.XQuantity, x)
yReal = Testlab.UnitSystem.MKSToUserValue(block.YQuantity, yReal)
yImag = Testlab.UnitSystem.MKSToUserValue(block.YQuantity, yImag)
yDB = Testlab.UnitSystem.ValueTodBValue(block.YQuantity, yAmp)
yPhase = Testlab.UnitSystem.MKSToUserValue(Testlab.UnitSystem.QuantityAngle, yPhase)
```

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Reading Property Data

Properties come in an AttributeMap of key-value pairs with multiple possible value types, such as System types of String or Integer, or LMSTestLabAutomation objects such as Enumerate, IScalar, or IScalarVariable, or as an IData such as a ::CName.

```
Dim props As LMSTestLabAutomation.AttributeMap
Dim weightEnum As LMSTestLabAutomation.Enumerate
Dim pointID As String
Set props = block.Properties
Set weightEnum = props.Item("Weighting")
pointID = props.Item("Point id")

MsgBox block.Label & vbNewLine & "Point ID: " & pointID & vbNewLine & _
  "Weight: " & weightEnum.LocalValue & vbNewLine & _
  "x = " & x & " " & Testlab.UnitSystem.Label(block.XQuantity) & vbNewLine & _
  "y (real, imag) = (" & yReal & ", " & yImag & ")" & vbNewLine & _
  "y (decibel, phase) = (" & yDB & ", " & yPhase & ")" & vbNewLine
```

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Creating new data

Modifications to an IBlock object initially exist only in the automation's memory, allowing calculations to be performed without unintentional changes to the source data. To save a result, the .AddItem method of the IDatabase allows the modified block to be written from the automation back into the project file.

```
Dim newBlock As LMSTestLabAutomation.IBlock2
Dim newYs() As Double
ReDim newYs(UBound(ysReal))
Dim point_three As Double
point_three = Testlab.UnitSystem.UserToMKSValue(block.YQuantity, 0.3)
For i = 0 To UBound(ysReal)
  newYs(i) = ysReal(i) + point_three
Next
Set newBlock = block.ReplaceYDoubleValues(newYs)
Call Project.AddItem(folder, blockName & "_Modified", newBlock, OverwriteIfExists:=1)
```

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Creating new data

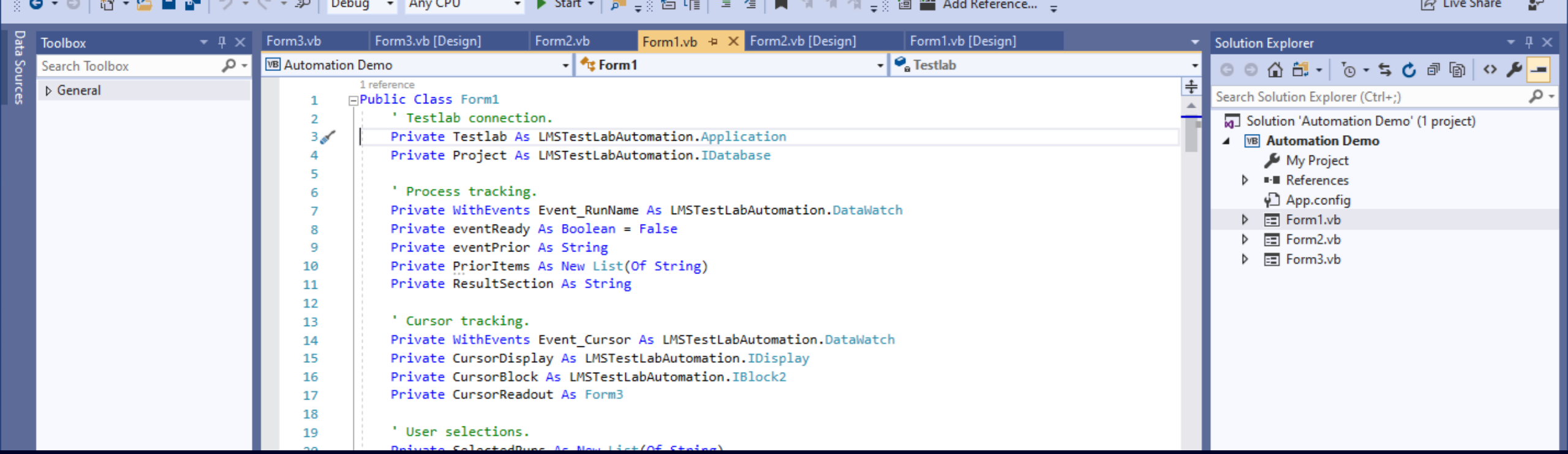
Some common calculations are already built into the Testlab automation library, and can be accessed through the ICommand object retrieved by the .cmd property.

```
Set newBlock = Testlab.cmd.
```

- AdvancedCmd
- BLOCK_ADD
- BLOCK_ADD_SCALAR
- BLOCK_ARRAY_STATISTICS
- BLOCK_ARTICULATION_INDEX
- BLOCK_AUTOPOWER_SPECTRUM
- BLOCK_AVERAGE_STATISTICS

```
Set newBlock = Testlab.cmd.BLOCK_ADD_SCALAR(block, Testlab.Factory.DOUBLE_TO_SCALAR(0.3, block.YQuantity))
Call Project.AddItem(folder, blockName & "_Add_Scalar", newBlock, OverwriteIfExists:=1)
```


Unrestricted | © Siemens 2025 | Siemens Digital Industries Software
SIEMENS



Useful Resources

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Useful Resources

Simcenter Testlab Automation:

- Forum and Knowledge Articles: https://community.sw.siemens.com/s/topic/0TO4O000000Mk87WAC/simcenter-testlab-automation
- Manuals and Object Model: https://community.sw.siemens.com/s/question/0D54O00006SDhh2SAD/two-useful-testlab-automation-documents

VBA for Microsoft Office Macros:

- https://learn.microsoft.com/en-us/office/vba/api/overview/language-reference

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

Thank you!

Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Contact

Published by Siemens DISW

Simcenter Testlab Support

Phone +1 800-955-0000

E-mail supporttest.lmsna.plm@siemens.com

Support Center https://support.sw.siemens.com/en-US/

Community Site https://community.sw.siemens.com/s/

 Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS